

UNIVERSIDADE PRESBITERIANA MACKENZIE
CIÊNCIA DA COMPUTAÇÃO

CAROLINA LEE
ENRIQUE CIPOLLA MARTINS
PEDRO HENRIQUE SARAIVA ARRUDA

PROJETO DE COMPILADORES

SÃO PAULO
2026

SUMÁRIO

ETAPA 1 - Gramática Livre de Contexto

1.1 PALAVRA RESERVADA

1.2 ESPECIFICAÇÃO DE TOKENS

1.3 ESTRUTURA DO PROGRAMA

1.4 DECLARAÇÃO

1.5 COMANDOS

1.6 EXPRESSÕES

STRUCTS FORNECIDOS PELA PROFESSORA

ESTRUTURAS INSERIDAS

ETAPA 2 - Analisador Léxico

ETAPA 3 - Analisador sintático

COMO EXECUTAR O CÓDIGO

TESTES DE SAÍDA

ETAPA 1 - Gramática Livre de Contexto

Segue as expressões regulares e gramática Livre de Contexto da linguagem Minivisualg

1.1 PALAVRAS RESERVADAS

As palavras reservadas de uma linguagem de programação são termos que possuem significado próprio na gramática e o desenvolvedor não deve usá-las para nomear variáveis, funções, classes, structs, etc.

As palavras reservadas mapeadas da linguagem Minivisualg são:

algoritmo, var, inicio, fimalgoritmo.

caractere, inteiro, real, logico,

verdadeiro, falso,

leia, escreva, escreval

se, entao, senao, fimse

para, de, ate, passo, faca, fimpara,

enquanto, fimenquanto,

vetor,

procedimento, fimprocedimento,

funcao, fimfuncao, retorne

MOD, E, OU

1.2 ESPECIFICAÇÃO DE TOKENS

id -> [a-zA-Z][a-zA-Z0-9_]*

num_int -> [0-9]+

num_real -> [0-9]+\.[0-9]+

cadeia -> "[^"]*"

comentario -> //[^\n]* (ignorado)

espaco_branco -> [\t\n]+ (ignorado)

opReal -> < | <= | = | <> | > | >=

atrib -> <-

opArit -> + | - | * | / | \

delimitador -> (|) | [|] | : | , | ..

1.3 ESTRUTURA DO PROGRAMA

algoritmo -> algoritmo cadeia declaracao* inicio comando* fimalgoritmo

declaracao -> declaracao_var | declaracao_procedimento | declaracao_funcao

1.4 DECLARAÇÃO

declaracao_var -> var declaracao_lista+

declaracao_lista -> id_lista ':' tipo

id_lista -> id (',' id)*

tipo -> tipo_base | vetor '[' num_int '..' num_int ']' de tipo_base

tipo_base -> inteiro | real | caractere | logico

declaracao_procedimento -> procedimento id '(' parametros ')'? inicio comando*
fimprocedimento

declaracao_funcao -> funcao id '(' parametros? ')' ':' tipo_base inicio comando*
fimfuncao

parametros -> parametro (',' parametro)*

parametro -> id ':' tipo_base

1.5 COMANDOS

comando -> atribuicao | leitura | escrita | condicional

 | repeticao_para | repeticao_enquanto | chamada | retorno

atribuicao -> variavel '<-' expressao

variavel -> id ('[' expressao '])'?

leitura -> leia '(' variavel ')'

escrita -> (escreva | escreval) '(' expressao (',' expressao)* ')'

condicional -> se '(' expressao ')' entao comando* (senao comando*)? fimse

repeticao_para -> para id de expressao ate expressao (passo expressao)? faca
comando* fimpara

repeticao_enquanto -> enquanto '(' expressao ')' faca comando* fimenquanto

chamada -> id '(' (expressao (',' expressao)*)? ')'?

retorno -> retorne expressao

1.6 EXPRESSÕES

```
expressao -> expressao_e (OU expressao_e)*
expressao_e -> expressao_rel (E expressao_rel)*
expressao_rel -> expressao_arit (opReal expressao_arit)?
expressao_arit -> termo (('+' | '-' ) termo)*
termo -> fator (('*' | '/' | '\' | MOD) fator)*
fator -> '(' expressao ')'
        | '-' fator
        | id ('[' expressao ']' | '(' (expressao (',' expressao)*)? ')')?
        | num_int | num_real | cadeia | verdadeiro | falso
```

STRUCTS FORNECIDOS PELA PROFESSORA

O código começa com a inclusão das bibliotecas, variáveis globais que foram adicionadas para o funcionamento do código e os structs fornecidos pela professora.

Estrutura do Token com a Union de Atributos

```
typedef struct {
    TokenNome type; // Nome do token
    int line;       // Para tratamento de erros

    // valor do atributo
    union {
        int table_index; // índice para Tabela de Símbolos
        int int_value;   // Valor literal convertido
        double float_value; // Valor literal convertido
        OpRelType op_code; // operador relacional específico
    } attribute;
} Token;
```

Definição dos nomes dos tokens (usados pelo Parser)

```
typedef enum {
    TOKEN_EOF = 0,
    TOKEN_ID, // Identificadores (variáveis, funções)
    TOKEN_NUM_INT, // Números inteiros (Ex: 42)
    TOKEN_NUM_FLOAT, // Números reais (Ex: 3.14)
    TOKEN_OP_REL, // Operadores relacionais
    TOKEN_KEYWORD // Palavras reservadas
} TokenNome;
```

Sub-códigos para especificar o atributo de operadores relacionais typedef enum {

```
    OP_LT, // < (Less Than)
    OP_LE, // <= (Less or Equal)
    OP_EQ, // == (Equal)
    OP_GT, // > (Greater Than)
    OP_GE, // >= (Greater or Equal)
} OpRelAtributo;
```

ESTRUTURAS INSERIDAS

As variáveis globais que foram adicionadas ao código e o que elas fazem:

lexemaAtual armazena a cadeia de caracteres até o último token, permitindo que o analisador sintático consulte o código original sem desvios.

arquivoSaida é apenas o ponteiro usado para gerar o arquivo de saída

tokenAtual é a variável que armazena o token retornado pela última chamada da função obterToken().

OP_ASSIGN é o operador de atribuição. Como a atribuição é formado por dois caracteres (< -), o programa lê o < e “espia” o próximo caractere: se for o -, ele junta os dois em um único comando de atribuição (OP_ASSIGN).

OP_NE é o operador diferente, <>, no Minivualg, segue a mesma lógica do anterior.

ETAPA 2 - Analisador Léxico

O analisador léxico é o responsável por transformar o código-fonte, neste caso em na linguagem MiniVisualAlg, em tokens.

Nosso código de analisador léxico funciona do seguinte modo:

A função iniciarAnalisador() abre o arquivo txt que contém o código-fonte e começa a monitorar a linhaAtual, enquanto a função fimDoArquivo() monitora o fim do arquivo. Em seguida, proximoToken realiza a limpeza da entrada, retirando caracteres sem valor semântico, como espaço em branco, quebra de linha, comentários, etc. Se o caractere encontrado for válido semanticamente determina-se é letra, sublinhado, número, aspas duplas, operadores relacionais/compostos, aritméticos, atribuição ou delimitadores.

Pega o token lido e verifica se é uma palavra chave, se não for atribui um tipo aquele token.

Por fim, trata erros léxicos e gera o arquivo de saída.

ETAPA 3 - Analisador sintático

O analisador sintático foi desenvolvido, de modo que cada regra da gramática mapeada na Etapa 1 do projeto tem uma função dedicada. O analisador sintático começa chamando o léxico quando precisa do próximo token, o que é feito por meio da função (analisadorSintatico()).

O analisador sintático tem funções auxiliares para conferir se o tipo recebido do token é o tipo esperado, se o token atual é uma palavra reservada, se é um delimitador ou um operador relacional. Em seguida, implementa as regras da gramática, para expressão, comandos, declarações e a estrutura do código-fonte. Por fim, trata erros sintáticos.

COMO EXECUTAR O CÓDIGO

Como compilar:

```
gcc -Wall -Wno-unused-result -g -Og compilador.c -o compilador
```

Como rodar, passando o arquivo fonte MiniVisualg como argumento:

```
./compilador <nome_do_arquivo>.txt
```

TESTES DE SAÍDA

```
algoritmo "nome_do_algoritmo"
var
    nome: caractere
    idade: inteiro
inicio
    // Seção de comandos e lógica
    escreva("Digite seu nome: ")
    leia(nome)
    escreva("Digite sua idade: ")
    leia(idade)

    escreva("Olá, ", nome, ". Você tem ", idade, " anos.")
finalgoritmo
```

```

1# PALAVRA_RESERVADA | 0
1# IDENTIFICADOR | 0
2# PALAVRA_RESERVADA | 0
3# IDENTIFICADOR | 1
3# OPERADOR_DELIMITADOR | 58
3# PALAVRA_RESERVADA | 0
4# IDENTIFICADOR | 2
4# OPERADOR_DELIMITADOR | 58
4# PALAVRA_RESERVADA | 0
5# PALAVRA_RESERVADA | 0
7# PALAVRA_RESERVADA | 0
7# OPERADOR_DELIMITADOR | 40
7# IDENTIFICADOR | 3
7# OPERADOR_DELIMITADOR | 41
8# PALAVRA_RESERVADA | 0
8# OPERADOR_DELIMITADOR | 40
8# IDENTIFICADOR | 4
8# OPERADOR_DELIMITADOR | 41
9# PALAVRA_RESERVADA | 0
9# OPERADOR_DELIMITADOR | 40
9# IDENTIFICADOR | 5
9# OPERADOR_DELIMITADOR | 41
10# PALAVRA_RESERVADA | 0
10# OPERADOR_DELIMITADOR | 40
10# IDENTIFICADOR | 6
10# OPERADOR_DELIMITADOR | 41
12# PALAVRA_RESERVADA | 0
12# OPERADOR_DELIMITADOR | 40
12# IDENTIFICADOR | 7
12# OPERADOR_DELIMITADOR | 44
12# IDENTIFICADOR | 8
12# OPERADOR_DELIMITADOR | 44
12# IDENTIFICADOR | 9
12# OPERADOR_DELIMITADOR | 44
12# IDENTIFICADOR | 10
12# OPERADOR_DELIMITADOR | 44
12# IDENTIFICADOR | 11
12# OPERADOR_DELIMITADOR | 41
13# PALAVRA_RESERVADA | 0

```

```
algoritmo "CalculadoraBasica"
var
    n1, n2: inteiro
inicio
    n1 <- 10
    n2 <- 3
    escreval("Soma: ", n1 + n2)
    escreval("Multiplicação: ", n1 * n2)
    escreval("Divisão real (/): ", n1 / n2)
    escreval("Divisão inteira (\): ", n1 \ n2)
finalgoritmo
```

```

1# PALAVRA_RESERVADA | 0
1# IDENTIFICADOR | 0
2# PALAVRA_RESERVADA | 0
3# IDENTIFICADOR | 1
3# OPERADOR_DELIMITADOR | 44
3# IDENTIFICADOR | 2
3# OPERADOR_DELIMITADOR | 58
3# PALAVRA_RESERVADA | 0
4# PALAVRA_RESERVADA | 0
5# IDENTIFICADOR | 3
5# OPERADOR_DELIMITADOR | 5
5# NUM_INTEIRO | 10
6# IDENTIFICADOR | 4
6# OPERADOR_DELIMITADOR | 5
6# NUM_INTEIRO | 3
7# PALAVRA_RESERVADA | 0
7# OPERADOR_DELIMITADOR | 40
7# IDENTIFICADOR | 5
7# OPERADOR_DELIMITADOR | 44
7# IDENTIFICADOR | 6
7# OPERADOR_DELIMITADOR | 43
7# IDENTIFICADOR | 7
7# OPERADOR_DELIMITADOR | 41
8# PALAVRA_RESERVADA | 0
8# OPERADOR_DELIMITADOR | 40
8# IDENTIFICADOR | 8
8# OPERADOR_DELIMITADOR | 44
8# IDENTIFICADOR | 9
8# OPERADOR_DELIMITADOR | 42
8# IDENTIFICADOR | 10
8# OPERADOR_DELIMITADOR | 41
9# PALAVRA_RESERVADA | 0
9# OPERADOR_DELIMITADOR | 40
9# IDENTIFICADOR | 11
9# OPERADOR_DELIMITADOR | 44
9# IDENTIFICADOR | 12
9# OPERADOR_DELIMITADOR | 47
9# IDENTIFICADOR | 13
9# OPERADOR_DELIMITADOR | 41
10# PALAVRA_RESERVADA | 0
10# OPERADOR_DELIMITADOR | 40
10# IDENTIFICADOR | 14
10# OPERADOR_DELIMITADOR | 44
10# IDENTIFICADOR | 15
10# OPERADOR_DELIMITADOR | 92
10# IDENTIFICADOR | 16
10# OPERADOR_DELIMITADOR | 41
11# PALAVRA_RESERVADA | 0

```

```

algoritmo "RotinasComRetorno"
funcao somar(a: inteiro, b: inteiro): inteiro
inicio
    retorne a + b // O processamento encerra aqui e devolve o valor
fimfuncao
var
    resultado: inteiro
inicio
    resultado <- somar(10, 5)
    escreval("O resultado retornado foi: ", resultado)
finalgoritmo

```

```

1# PALAVRA_RESERVADA | 0
1# IDENTIFICADOR | 0
2# PALAVRA_RESERVADA | 0
2# IDENTIFICADOR | 1
2# OPERADOR_DELIMITADOR | 40
2# IDENTIFICADOR | 2
2# OPERADOR_DELIMITADOR | 58
2# PALAVRA_RESERVADA | 0
2# OPERADOR_DELIMITADOR | 44
2# IDENTIFICADOR | 3
2# OPERADOR_DELIMITADOR | 58
2# PALAVRA_RESERVADA | 0
2# OPERADOR_DELIMITADOR | 41
2# OPERADOR_DELIMITADOR | 58
2# PALAVRA_RESERVADA | 0
3# PALAVRA_RESERVADA | 0
4# PALAVRA_RESERVADA | 0
4# IDENTIFICADOR | 4
4# OPERADOR_DELIMITADOR | 43
4# IDENTIFICADOR | 5
5# PALAVRA_RESERVADA | 0
6# PALAVRA_RESERVADA | 0
7# IDENTIFICADOR | 6
7# OPERADOR_DELIMITADOR | 58
7# PALAVRA_RESERVADA | 0
8# PALAVRA_RESERVADA | 0
9# IDENTIFICADOR | 7
9# OPERADOR_DELIMITADOR | 5
9# IDENTIFICADOR | 8
9# OPERADOR_DELIMITADOR | 40
9# NUM_INTEIRO | 10
9# OPERADOR_DELIMITADOR | 44
9# NUM_INTEIRO | 5
9# OPERADOR_DELIMITADOR | 41
10# PALAVRA_RESERVADA | 0
10# OPERADOR_DELIMITADOR | 40
10# IDENTIFICADOR | 9
10# OPERADOR_DELIMITADOR | 44
10# IDENTIFICADOR | 10
10# OPERADOR_DELIMITADOR | 41
11# PALAVRA_RESERVADA | 0

```

```

5 texto.txt
1  algoritmo "ProcedimentosSemParametros"
2  // Você os declara antes do início principal do programa
3  // Procedimentos que não pedem nenhum parâmetro não devem ter
4  // parênteses na sua declaração, nem na hora de invocá-los!
5  procedimento linha_decorativa
6  início
7  escreval("-----")
8  fimprocedimento

```

```

10 PALAVRA_RESERVADA | 0
10 IDENTIFICADOR | 0
50 PALAVRA_RESERVADA | 0
50 IDENTIFICADOR | 1
60 PALAVRA_RESERVADA | 0
70 PALAVRA_RESERVADA | 0
70 OPERADOR_DELIMITADOR | 40
70 IDENTIFICADOR | 2
70 OPERADOR_DELIMITADOR | 41
80 PALAVRA_RESERVADA | 0

```

```

algoritmo "ListaNomes"
var
  nomes: vetor[1..3] de caractere
inicio
  nomes[1] <- "Ana"
  nomes[2] <- "Carlos"
  nomes[3] <- "João"
  escreval("A pessoa na posição 2 é: ", nomes[2])
finalgoritmo

```

```

1# PALAVRA_RESERVADA | 0
1# IDENTIFICADOR | 0
2# PALAVRA_RESERVADA | 0
3# IDENTIFICADOR | 1
3# OPERADOR_DELIMITADOR | 58
3# PALAVRA_RESERVADA | 0
3# OPERADOR_DELIMITADOR | 91

```

```

algoritmo "ContagemSimples"
var
  i: inteiro
inicio
  escreval("Iniciando as voltas...")
  para i de 1 ate 5 faca
    escreval("Completo a volta número: ", i)
  fimpara
finalgoritmo

```

```

1# PALAVRA_RESERVADA | 0
1# IDENTIFICADOR | 0
2# PALAVRA_RESERVADA | 0
3# IDENTIFICADOR | 1
3# OPERADOR_DELIMITADOR | 58
3# PALAVRA_RESERVADA | 0
4# PALAVRA_RESERVADA | 0
5# PALAVRA_RESERVADA | 0
5# OPERADOR_DELIMITADOR | 40
5# IDENTIFICADOR | 2
5# OPERADOR_DELIMITADOR | 41
6# PALAVRA_RESERVADA | 0
6# IDENTIFICADOR | 3
6# PALAVRA_RESERVADA | 0
6# NUM_INTEIRO | 1

```

```

algoritmo "ParqueDeDiversoes"
var
    idade: inteiro
    altura: real
    podeBrincar: logico
inicio
    idade <- 14
    altura <- 1.60
    // O E exige que os DOIS lados sejam verdadeiros
    // O OU, basta um ser verdadeiro
    podeBrincar <- (idade >= 12) E (altura >= 1.50)
    escreval("Pode entrar na Montanha Russa? ", podeBrincar)
fimalgoritmo

```

```
1# PALAVRA_RESERVADA | 0
1# IDENTIFICADOR | 0
2# PALAVRA_RESERVADA | 0
3# IDENTIFICADOR | 1
3# OPERADOR_DELIMITADOR | 58
3# PALAVRA_RESERVADA | 0
4# IDENTIFICADOR | 2
4# OPERADOR_DELIMITADOR | 58
4# PALAVRA_RESERVADA | 0
5# IDENTIFICADOR | 3
5# OPERADOR_DELIMITADOR | 58
5# PALAVRA_RESERVADA | 0
6# PALAVRA_RESERVADA | 0
7# IDENTIFICADOR | 4
7# OPERADOR_DELIMITADOR | 5
7# NUM_INTEIRO | 14
8# IDENTIFICADOR | 5
8# OPERADOR_DELIMITADOR | 5
8# NUM_REAL | 1.60
11# IDENTIFICADOR | 6
11# OPERADOR_DELIMITADOR | 5
11# OPERADOR_DELIMITADOR | 40
11# IDENTIFICADOR | 7
11# OPERADOR_DELIMITADOR | 4
11# NUM_INTEIRO | 12
11# OPERADOR_DELIMITADOR | 41
11# PALAVRA_RESERVADA | 0
11# OPERADOR_DELIMITADOR | 40
11# IDENTIFICADOR | 8
11# OPERADOR_DELIMITADOR | 4
11# NUM_REAL | 1.50
11# OPERADOR_DELIMITADOR | 41
12# PALAVRA_RESERVADA | 0
12# OPERADOR_DELIMITADOR | 40
12# IDENTIFICADOR | 9
12# OPERADOR_DELIMITADOR | 44
12# IDENTIFICADOR | 10
12# OPERADOR_DELIMITADOR | 41
13# PALAVRA_RESERVADA | 0
```